

Decentralised Plan Repair for a Multi-Robot Warehouse under Dynamic Disruptions

Autonomous Assignment Report

Group members

Name	Roll No.
Ekta	B23CS1018
Tavishi Srivastava	B23CS1101
Shreekar	B23CS1069

1. Problem statement

A warehouse is modelled as a 2D grid with static shelves. Each robot starts at a cell and must visit, in order, a list of pickup and drop-off locations. Initial collision-free paths come from a multi-agent path-finding algorithm (Space-Time A*). During execution the environment changes: a cell can be blocked, a robot can break down, or a robot can receive an urgent task. The robots must repair their plans **without re-invoking the global planner**, and the repair should change the plans of as few robots as possible. We evaluate (i) the total time steps needed by all agents, (ii) the number of agents whose plan is changed per disruption, and (iii) how both vary with the number of agents and the density of dynamic obstacles. We also identify settings where the system fails.

2. System design

The implementation is in Python and split into modules: `grid.py` (map and warehouse layout), `planner.py` (Space-Time A* and reservation table), `repair.py` (plan repair and negotiation), `sim.py` (time-step simulator, disruptions, metrics), `viz.py` (GIF animation), `viz_html.py` (interactive viewer) and `run_experiments.py`.

2.1 Environment

The default map is 20x20 with shelf blocks (2x4 cells) separated by aisles of width 2; width 1 is used as a failure setting. Robots move one cell per step in 4 directions or wait. Each robot has 2 tasks (pickup next to a shelf, drop-off on the border), so 4 waypoints. A robot that finishes parks on its last cell and remains an obstacle for the others.

2.2 Space-Time A* with reservations

A search state is (cell, time). Moves are the four directions plus wait. The heuristic is the true shortest-path distance on the static map (BFS), which is admissible. A move to (c, t+1) is rejected if the reservation table says that (i) another agent occupies c at t+1 (vertex conflict), (ii) another agent moves c to the current cell during the same step (swap/edge conflict), (iii) a robot is parked on c from a time not later than t+1, or (iv) a dynamic obstacle blocks c at t+1. For the last waypoint the agent parks, so the goal is only accepted after every other reservation on that cell has expired. Initial plans use prioritised planning: agent 0 plans first and reserves its path, agent 1 plans around it, and so on.

2.3 Disruptions

- **Blocked cell:** a free, unoccupied cell becomes impassable for a random duration (5 to 15 steps). Agents whose plans use the cell during that interval are the *affected* agents.
- **Robot breakdown:** a robot freezes in its cell for N steps (or permanently). Its cell is reserved as occupied; agents whose plans pass through it are affected. After recovery the robot re-plans.

- **Emergency task:** an extra waypoint is inserted at the front of a robot's task list; the robot must re-plan to visit it first.

Dynamic obstacle *density* is defined as the expected fraction of free cells that is blocked at any moment. Block events are generated from this value (number of events = density x free cells x horizon / mean duration), and breakdowns and emergencies are added at fixed counts (2 each in the main sweep).

3. Plan repair and negotiation

When a disruption occurs, only the affected agents start a repair. A repair never calls the multi-agent planner; every agent keeps its current plan unless the repair explicitly changes it. Each disrupted agent climbs an **escalation ladder**:

- **Level 1 - self repair:** remove the agent's own reservations and re-plan only that agent against everybody else's unchanged plans. If it succeeds, exactly one agent has changed.
- **Level 2 - negotiate with one neighbour:** agents within Manhattan distance r (default 8) are candidates. For each candidate the disrupted agent plans first, then the candidate *yields*, i.e. re-plans its remaining route around the new path. The option with the least extra delay is committed.
- **Level 3 - two neighbours** (pairs among the 5 nearest) and **level 4 - the k nearest neighbours** (k up to 3).
- **Failure - hold:** the agent stays in place (its cell is reserved) and retries every 4 steps. If it is still stuck after 30 steps it is declared failed.

A neighbour that is asked to yield can also be a robot that has already finished; it then has to leave and come back to its cell. If a forced hold overwrites someone else's reservation, that agent is detected as displaced and repaired in turn (cascade). In the report, the number of *agents changed* for one disruption is computed by comparing every agent's remaining path before and after the event (trailing waits ignored).

```
repair(disrupted agents A):
  for a in A (by id):
    if replan_alone(a):                # level 1
      continue
    for k in 1..max_group:             # levels 2..4
      best = None
      for group G of k neighbours within radius r:
        plans = trial(a, G)            # a plans first, then each g in G re-plans
        if plans ok and cost(plans) < best.cost: best = (G, plans)
      if best: commit(best); break
    else:
      hold(a)                          # retry later; may displace others
  re-run repair for agents displaced by a hold (max 3 times each)
```

4. Experimental setup

All experiments use a 20x20 warehouse with aisle width 2, 2 tasks per agent, 2 breakdowns (duration 6 to 20 steps) and 2 emergency tasks per run, and a maximum of 400 steps. **Experiment A** varies the number of agents (5, 10, 15, 20, 25) and obstacle density (0, 2, 4, 6, 8%) with 10 random seeds per cell, for our local repair and for a **baseline** that, on every disruption, discards all plans and re-plans every agent from scratch (used only for comparison). Both modes see identical events for the same seed. **Experiment B** deliberately stresses the system (10 seeds each). Metrics: **total time steps** = sum over agents of the step at which each finishes its last task (reported over successful runs); **agents changed per disruption** averaged over disruptions that affected at least one agent; **success** = all agents completed all tasks. A built-in collision check (vertex, swap, entering a blocked cell) reported 0 collisions in all runs reported here.

5. Visualisation

`run_demo.py` produces two outputs from the same simulation run. `demo.gif` is an animation, and `demo.html` is an interactive viewer that opens in any web browser and supports pause, about 10 seconds backward/forward, single steps, a time slider and jumping to each disruption. The warehouse is drawn as a labelled 2D grid (x across, y down): shelves are dark grey, blocked cells are red with a white cross and their (x,y) coordinates, and a freshly blocked cell gets a gold outline. Every robot shows its ID inside its cell, and a side panel lists the cell of every robot at every step. Planned paths stay visible; a robot that was just re-planned gets an orange ring and its previous plan is drawn dashed, so the repair can be compared with the original plan. The screenshot below is from the scripted demo (8 robots).

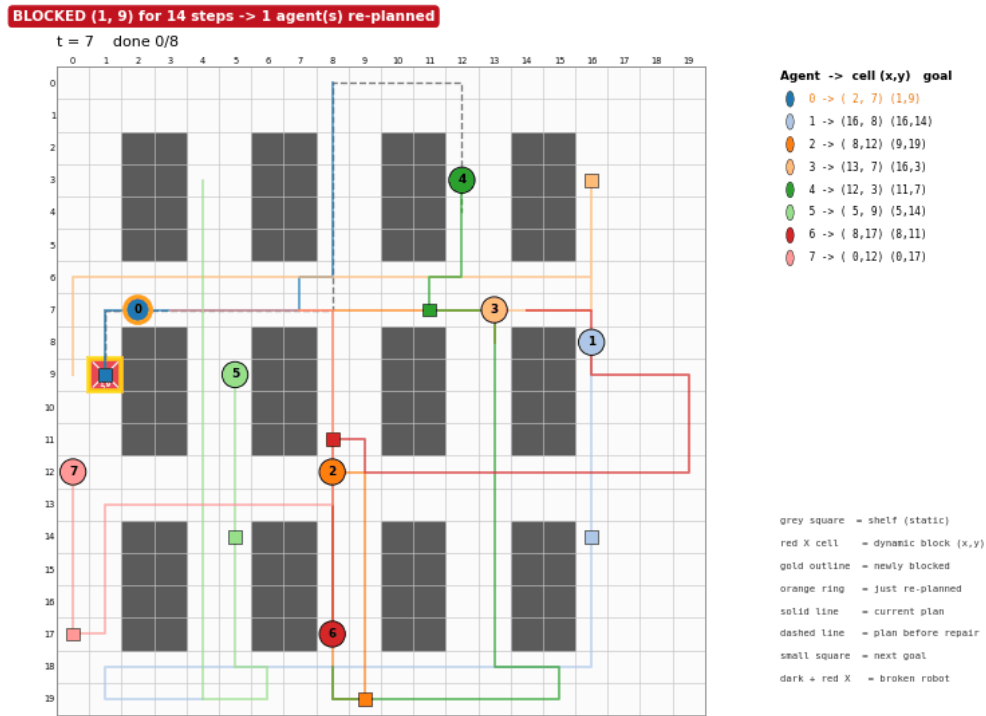


Figure 1: a cell, (1,9), is blocked at $t=6$ (red, gold outline). Only robot 0, whose path crossed it, re-plans (orange ring); its old plan is the dashed line.

6. Results

6.1 Effect of the number of agents and obstacle density

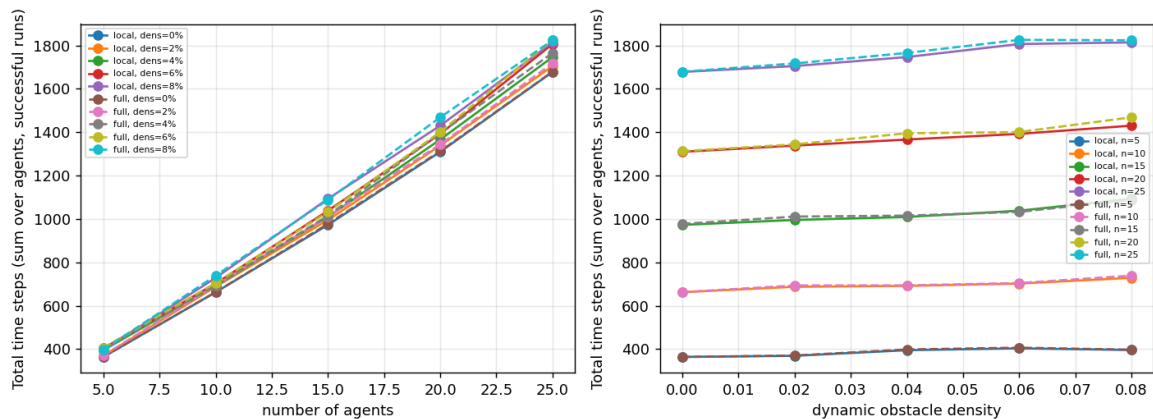


Figure 2: total time steps (sum over agents, successful runs). Solid = local repair, dashed = replan-everything baseline.

Agent s	Density	Success local	Total steps local	Total steps full	Agents changed local	Agents changed full	Max changed local / full
5	0%	10/10	364	364	0.93	2.22	2 / 4
5	4%	10/10	395	398	1.09	1.63	2 / 4
5	8%	10/10	396	397	1.05	1.51	2 / 4
10	0%	10/10	662	662	1.07	2.97	2 / 7
10	4%	9/10	691	694	1.19	2.37	3 / 8
10	8%	10/10	728	739	1.19	2.31	4 / 9
15	0%	10/10	973	977	0.92	4.17	5 / 10
15	4%	9/10	1009	1015	1.19	3.41	4 / 15
15	8%	10/10	1093	1088	1.24	3.18	6 / 13
20	0%	10/10	1310	1313	1.47	6.15	3 / 17
20	4%	10/10	1366	1395	1.36	4.47	4 / 18
20	8%	10/10	1430	1469	1.30	4.16	7 / 19
25	0%	10/10	1678	1679	1.27	7.50	3 / 23
25	4%	10/10	1746	1766	1.40	5.11	5 / 25
25	8%	10/10	1814	1824	1.41	5.05	6 / 25

Table 1: selected results (full grid in results/sweep.csv). 'Agents changed' = mean over disruptions that affected at least one agent.

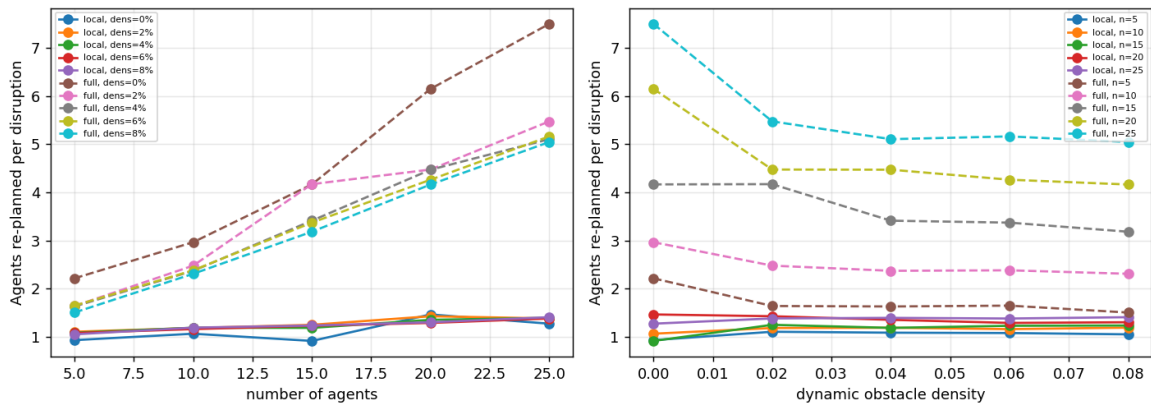


Figure 3: agents whose plan changed per disruption.

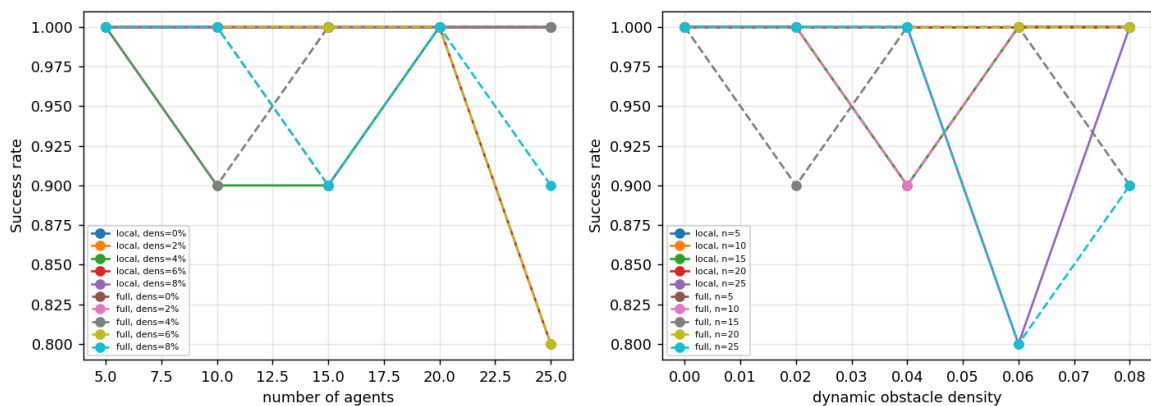


Figure 4: success rate (fraction of runs in which every agent finished).

Observations

- **Plan changes are small and do not grow with fleet size under local repair.** The local repair re-plans about 1.1 agents per affected disruption with 5 agents and 1.4 with 25 (4% density). The replan-everything baseline grows from 1.6 to 5.1, roughly proportional to the fleet, with a maximum of up to 25 agents in a single disruption versus at most 7 for local repair in the whole sweep.
- **Total time steps are essentially the same for both strategies** (differences of a few percent, in either direction), so keeping most plans untouched costs no measurable efficiency in these experiments. The baseline does however spend more computation: its repair CPU time per run is larger, e.g. about 0.45 s versus 0.06 s at 20 agents and 8% density (sweep.csv, column repair_cpu).
- **Total time grows roughly linearly with the number of agents** (about 66 steps per added agent), because it is a sum over agents.
- **Obstacle density increases total time moderately.** Going from 0% to 8% adds roughly 30 to 140 extra steps in total for 5 to 25 agents (measured against the initial plan cost, the 'delay' column in sweep.csv). Part of the delay at 0% (about 40 to 45 steps) comes from the breakdowns and emergency tasks, which are present in every run.
- **Success rate stays high** (at least 8 of 10 in every cell) but drops occasionally for 10 to 25 agents at 4 to 8% density in both modes.

6.2 Failure analysis: settings where agents fail to complete their tasks

We searched for breaking points by stressing one factor at a time (10 seeds per setting, 20x20 map unless stated, 2 breakdowns and 2 emergencies unless stated). 'Stuck' means a robot could not find any plan, even with negotiation, for 30 consecutive steps. 'Initial plan' means prioritised planning found no path for that robot at time 0.

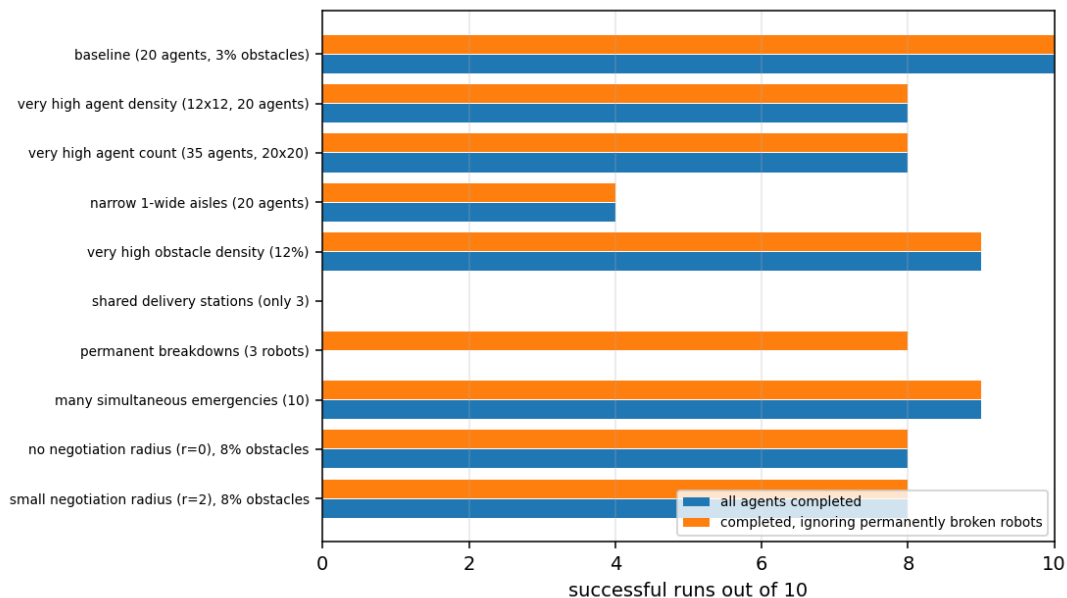


Figure 5: success counts for the stress settings.

Setting	Completed	Agents done (mean fraction)	Agents changed	Failure reasons (counts of agents over all runs)
baseline (20 agents, 3% obstacles)	10/10	1.0	1.35	{}
very high agent density (12x12, 20 agents)	8/10	0.94	1.82	{initial_plan: 2, stuck: 10}
very high agent count (35 agents, 20x20)	8/10	0.99	1.62	{stuck: 2}

Setting	Completed	Agents done (mean fraction)	Agents changed	Failure reasons (counts of agents over all runs)
narrow 1-wide aisles (20 agents)	4/10	0.94	1.45	{stuck: 7, initial_plan: 5}
very high obstacle density (12%)	9/10	0.99	1.19	{stuck: 1}
shared delivery stations (only 3)	0/10	0.16	0.6	{initial_plan: 121, stuck: 5}
permanent breakdowns (3 robots)	0/10	0.85	2.29	{permanent_breakdown: 20, stuck: 2}
many simultaneous emergencies (10)	9/10	0.99	0.96	{stuck: 1}
no negotiation radius ($r=0$), 8% obstacles	8/10	0.99	1.25	{stuck: 2}
small negotiation radius ($r=2$), 8% obstacles	8/10	0.99	1.24	{stuck: 2}

Table 2: failure experiments (results/failures.csv).

Why the failures happen

- **Shared delivery stations (0/10).** With only 3 stations, the first robot that finishes parks on one, and every later robot that needs it can never deliver (121 agent-level initial-plan failures). This is the 'goal cell occupied' case: a parked robot is a permanent obstacle. A fix would be to let finished robots leave the grid or move to a waiting area.
- **Narrow 1-wide aisles (4/10).** Two robots meeting head-on in a single-width corridor cannot pass, and waiting does not help. Prioritised planning also fails at time 0 for some agents (5 initial-plan failures) because earlier robots park in or reserve the only route. Several robots then end up stuck.
- **Dense maps (12x12 with 20 agents: 8/10; 35 agents on 20x20: 8/10).** With many robots there is too little free space for a conflict-free path within the search horizon, and stuck robots hold their cell, which blocks others in turn. We did not diagnose each stuck robot individually, so we describe the cause as congestion consistent with these configurations rather than proven per case.
- **Permanent breakdowns (0/10, but 8/10 for the remaining robots).** By definition the broken robots never finish. The other robots mostly route around them, but in 2 cases a robot also became stuck (a dead robot can sit in a cell that others need).
- **High obstacle density (12%: 9/10), many emergencies (9/10).** Both stay mostly robust; the single failure in each case is again a stuck robot.
- **Small negotiation radius ($r=0$ or 2: 8/10 at 8% density).** Success and agents changed were the same as with the default radius in our runs, so we could not show a benefit of negotiation in this setting. Level 1 (self repair) resolves almost all disruptions here, so negotiation is rarely needed; it matters only in congested cases, where it also often fails.

7. Limitations and possible improvements

- Prioritised planning is incomplete: it can fail for some priority orders. Retrying with other orders, or a complete method such as CBS, would remove some initial-plan failures.
- Finished robots stay parked, which is the main cause of the station-contention failures. Letting them leave the floor would help.
- Stuck robots do not coordinate beyond the distance-based neighbour ladder. Deadlock detection and explicit swapping or back-off protocols would help in narrow aisles.
- Neighbours are chosen by Manhattan distance only; choosing agents whose plans actually conflict could find better negotiation partners.
- Negotiation is simulated centrally (one process evaluates each neighbour's reply with the shared reservation table); a real deployment would exchange messages.

- Results are for one map size, 2 tasks per agent and 10 seeds per cell, so small differences should not be over-interpreted.

8. Conclusion

A repair scheme that first re-plans only the disrupted agent and escalates to nearby neighbours only when needed kept the number of re-planned agents per disruption at about one to two regardless of fleet size, while matching the total time steps of replanning everything and using less computation. The system failed in clearly identifiable conditions: contested delivery cells, single-width aisles, very crowded maps and permanent breakdowns.

Appendix: how to reproduce

`python run_demo.py` produces `demo.gif`. `python run_experiments.py` reproduces `sweep.csv`, `failures.csv` and the plots. Seeds are fixed, so runs are reproducible.